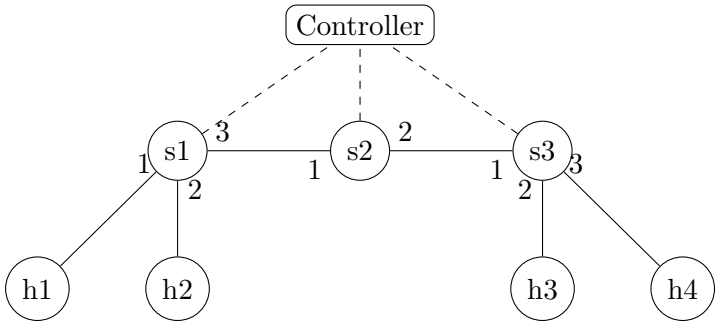
Piattaforme Software per la Rete (Modulo 1) – 14/07/2014

Su ogni foglio consegnato scrivere nome, cognome, matricola e il numero dell'esercizio.

Esercizio 1

Si consideri la rete in figura. I nodi h1,..., hn contengono tabelle ARP prepopolate per tutta la rete. Gli indirizzi MAC e IP dei nodi h1,..., hn sono MAC1,..., MACn e IP1,..., IPn. Sui nodi h1,..., hn sono in esecuzione processi in ascolto su tutte le porte TCP e UDP di interesse.



Il controllore POX è collegato ai nodi s1,..., sm ed è configurato come indicato nel Programma 1.

Scrivere tutti i pacchetti che sono inviati su tutti i collegamenti della rete e tra i nodi e il controllore in coseguenza degli eventi seguenti. Scrivere il contenuto delle tabelle openflow prima e dopo ogni evento.

(a) Apertura di una connessione TCP da h2 a h3 dalla porta 12345 alla porta 80;

(b) Invio di un datagramma UDP da h1 a h4 dalla porta 5432 alla porta 53 e relativa risposta;

(c) Invio di un nuovo datagramma UDP da h1 a h3 dalla porta 5432 alla porta 53 e relativa risposta.

Programma 1

```
from pox.core import core
import pox.openflow.libopenflow_01 as of

def _handle_ConnectionUp(event):
  if (event.dpid == 2): #event.dpid contiene il numero dello switch
    msg = of.ofp_flow_mod()
    msg.match.dl_src = MAC3
    msg.match.nw_proto = 6 # TCP
    msg.actions.append(of.ofp_action_output(2))
    event.connection.send(msg)
    msg = of.ofp_flow_mod()
    msg.match.dl_src = MAC2
    msg.match.nw_proto = 6 # TCP
    msg.actions.append(of.ofp_action_output(1))
    event.connection.send(msg)
  if (event.dpid == 3): #event.dpid contiene il numero dello switch
    msg = of.ofp_flow_mod()
    msg.match.dl_dst = MAC3
    msg.match.nw_proto = 17 # UDP
    msg.actions.append(of.ofp_action_dl_addr.set_dst(MAC4))
    msg.actions.append(of.ofp_action_nw_addr.set_dst(IP4))
    msg.actions.append(of.ofp_action_output(3))
    event.connection.send(msg)
    msg = of.ofp_flow_mod()

def _handle_PacketIn(event):
  packet = event.parsed
  if (event.dpid == 1) and (packet.dst == MAC3 or packet.dst == MAC4):
    msg = of.ofp_flow_mod()
    msg.match.dl_src = packet.src
    msg.match.dl_dst = packet.dst
    msg.match.nw_proto = 17 #UDP
    msg.actions.append(of.ofp_action_output(3))
    msg.buffer_id = event.ofp.buffer_id # applica subito
    event.connection.send(msg)
    return
  msg = of.ofp_packet_out(data = event.ofp)
  msg.actions.append(of.ofp_action_output(of.OFPP_FLOOD))
  event.connection.send(msg)

def launch():
  core.openflow.addListenerByName("ConnectionUp", _handle_ConnectionUp)
  core.openflow.addListenerByName("PacketIn", _handle_PacketIn)
```

Esercizio 2

Si consideri un filtro di Bloom costituito da un array di 7 bit e 2 funzioni hash. Inizialmente tutti i bit dell'array sono posti a zero. Si assuma che ciascuna posizione all'interno dell'array può essere selezionata da ciascuna funzione di hash con la medesima probabilità.

(a) Calcolare la probabilità che un dato bit sia posto ad 1 dopo aver effettuato l'inserimento di 3 elementi

(b) Calcolare la probabilità di un falso positivo.

(c) Siano le due funzioni di hash definite come $H_1(x) = 3x - 1 \mod 7$ e $H_2(x) = x + 2 \mod 7$. Inizialmente tutti i bit dell'array sono posti a zero. Inserire 1 e 4 nel filtro di Bloom. Mostrare lo stato di ogni bit prima e dopo ogni inserimento.

(d) Conclusi gli inserimenti, verificare l'appartenza al dataset degli elementi 7 e 2 e commentare i risultati ottenuti (vi sono falsi positivi?)

Esercizio 3

L'indirizzo 131.175.12.1 è associato a un server virtuale gestito tramite un load balancer che distribuisce il carico verso 8 server fisici. Per ogni richiesta di servizio il client apre quattro connessioni TCP verso il server virtuale. Le porte lato client sono scelte in ordine progressivo, mentre la porta lato server è sempre la 80. Tutti i server fisici possono svolgere il servizio richiesto, ma ogni servizio deve essere svolto integralmente dallo stesso server.

Nel load balancer è in esecuzione un algoritmo che funziona come segue: per ogni pacchetto entrante si calcola il seguente hash:

$$d = h(\text{IP src} \| \text{IP dst} \| \text{port src}/4)$$

Gli ultimi tre bit di d sono usati come selettore del server fisico.

(a) Dire se l'algoritmo definito sopra funziona correttamente in ogni situazione. Eventualmente proporre delle modifiche.

(b) Si consideri il caso in cui un sistema NPAT alteri i numeri porta lato client. Dire se l'algoritmo definito sopra funziona ancora correttamente ed eventualmente modificarlo per gestire la nuova situazione.

(c) Considerando un link di capacità $C = 10\,\text{Gbit/s}$, definire un meccanismo, anche probabilistico, che salvi su disco almeno un pacchetto per ogni relazione client-server che genera 25 Mbit/s o più.